SRM UNIVERSITY, AP

Department of Computer Science and Engineering

Comprehensive Technical Project Report

ConveneHub

Event Management, Ticketing & Attendance Platform

A minimal, role-based full-stack system for modern event operations

Project Focus

Secure event publishing, booking workflows, payment integration, QR-based verification, and operational dashboards.

Academic Year	2025-2026
Project Category	Full-Stack Web Application
Submitted To	Department of Computer Science and Engineering

Submitted By

Rohan Kumar Mohanta	AP24110011046
Aditya Pratap Singh	AP24110011050
Preetimohan Pratihari	AP24110011055
Aryan Bharmauria	AP24110011078
Devraj Sahani	AP24110011150

Contents

Executive Summary . . . **iii**

Acknowledgments . . . **iii**

1 Introduction . . . **1**
- 1.1 Scenario Based Case Study . . . 1
  - 1.1.1 Meet Sarah . . . 1
  - 1.1.2 Sarah's Experience with ConveneHub . . . 1
  - 1.1.3 Results & Impact . . . 2
  - 1.1.4 Key Takeaways . . . 2
- 1.2 System Requirements . . . 2
  - 1.2.1 Software Requirements . . . 2
  - 1.2.2 Hardware Requirements . . . 2

2 Project Architecture . . . **3**
- 2.1 Technical Architecture . . . 3
- 2.2 ER Diagram . . . 4
- 2.3 Key Features . . . 4
- 2.4 Roles and Responsibilities . . . 5
  - 2.4.1 User (Attendee) . . . 5
  - 2.4.2 Organizer . . . 5
  - 2.4.3 Promoter . . . 5
  - 2.4.4 Admin (Platform Administrator) . . . 6

3 User Flow . . . **6**

4 MVC Pattern . . . **7**
- 4.1 Model Layer . . . 8
- 4.2 Controller Layer . . . 8
- 4.3 View Layer . . . 8
- 4.4 Advantages of Using MVC in ConveneHub . . . 8
- 4.5 Project Structure in ConveneHub . . . 8

5 Project Setup and Configuration . . . **8**
- 5.1 Client Setup (Installing React App) . . . 8
- 5.2 Server Setup (npm init) . . . 9
- 5.3 Summary . . . 9

6 Backend Development . . . **9**
- 6.1 Backend Reference Video . . . 10
- 6.2 Backend Structure . . . 10

7 Database Development . . . **12**
- 7.1 Configure MongoDB . . . 12
- 7.2 Create Database Connection . . . 12
- 7.3 Create Schemas and Models . . . 13
- 7.4 Database Structure in ConveneHub . . . 14
- 7.5 Advantages of This Database Design . . . 14

8 Front-End Development . . . **14**
- 8.1 Frontend Structure . . . 14

9 Project Resources . . . **17**
- 9.1 Deployment and Repository Links . . . 17
- 9.2 Mock Data Reference . . . 17

10 Visual References **18**

Bibliography **22**

A Appendix **22**

A.1 Database Schema Reference 22

A.1.1 User Collection 22

A.1.2 Event Collection 23

A.1.3 Booking Collection 23

A.2 Environment Configuration Example 24

A.3 Postman Collection Snippet 24

Executive Summary

ConveneHub is a comprehensive, role-based event management and ticketing platform designed to streamline the entire event lifecycle from creation to attendee check-in. The platform provides a sophisticated multi-role system supporting organizers, attendees, promoters, and administrators with distinct workflows and capabilities tailored to their specific needs.

The system is built as a modern monorepo utilizing React with Vite for the frontend, Express.js for the backend API, and MongoDB with Mongoose for persistent data storage. The architecture emphasizes role-based access control (RBAC), financial settlement tracking, referral commission management, and seamless payment integration through Razorpay.

This report documents the technical architecture, implementation details, design patterns, technology choices, deployment strategy, and future enhancement opportunities for the platform. The project is positioned as production-ready, with Docker containerization and cloud deployment options such as Railway and Vercel.

Key Statistics

- 13 core data models with role-aligned collections.
- 35+ REST API endpoints supporting multi-role workflows.
- Role-based access control with 4 primary user roles.
- Razorpay payment integration for paid ticketing.
- QR code generation and verification for check-in.
- Referral and commission tracking for promoters.
- Multi-tenant support for organizational segregation.

Acknowledgments

This project represents a collaborative effort across design, development, and quality assurance. Special recognition goes to the development team for building robust backend services, the frontend team for creating a clear multi-role user experience, the QA team for validating end-to-end flows, and the DevOps contributors for deployment and monitoring readiness.

1 Introduction

1.1 Scenario Based Case Study

1.1.1 Meet Sarah

Sarah is an event organizer who manages multiple conferences and professional workshops. Before adopting ConveneHub, her business relied on fragmented tools, spreadsheets, manual commission calculations, and inefficient attendee check-in processes. As event volume increased, these disconnected workflows created operational delays, reduced visibility, and made it difficult to scale efficiently.

When Sarah discovered ConveneHub, she adopted the platform to centralize event publishing, registrations, promoter coordination, ticketing, payments, and on-site verification into a single system.

1.1.2 Sarah's Experience with ConveneHub

Sarah registered as an **Organizer** and quickly experienced the practical advantages of the platform across the full event lifecycle.

Event Creation: She created her first event with multiple ticket tiers within minutes, replacing a slower manual setup process.

Dashboard & Analytics: Real-time sales tracking, attendee insights, and revenue metrics became available from a single dashboard, saving more than five hours per week previously spent gathering data from multiple sources.

Attendee Management: Automated registrations, confirmations, and email notifications replaced spreadsheet-based tracking and reduced the risk of administrative errors.

Promoter Commission Tracking: Sarah managed referral links for five promoters with automatic commission calculations. A task that previously required nearly four hours per event was reduced to approximately thirty minutes, while promoter-led sales improved by 35%.

QR Code Tickets: The platform automatically generated secure QR code tickets, enabling convenient and reliable digital distribution.

Fast Check-in: Mobile QR scanning reduced attendee check-in time from about 45 seconds to 10 seconds per person, significantly improving on-site flow.

Revenue Insights: Data-driven analytics helped Sarah identify the most effective ticket tiers and promoter partnerships, contributing to a 25% increase in profit margins.

Payment Processing: Integrated online payment handling reduced manual processing effort and minimized reconciliation errors.

Multi-Event Scaling: By the end of the year, Sarah was managing four concurrent events with 200–500 attendees each, all from a unified operational dashboard.

1.1.3 Results & Impact

Metric	Improvement
Event management time	50–60% reduction
Commission calculation	4 hours → 30 minutes per event
Check-in speed	45 sec → 10 sec per attendee
Events managed per year	2–3 → 4–6
Promoter sales	+35% increase
Profit margins	+25% increase

1.1.4 Key Takeaways

ConveneHub enables event organizers to:

- **Eliminate fragmentation** by consolidating ticketing, attendee management, and commission workflows into a single platform.
- **Automate repetitive tasks** such as ticket generation, notifications, and commission calculations.
- **Access real-time analytics** to support faster and more informed decision-making.
- **Scale efficiently** without requiring proportional growth in manual operational effort.

> "ConveneHub transformed my business. I went from drowning in spreadsheets to managing 4 events with ease. Commission tracking is flawless, promoters are happier, and I can focus on creating amazing events."
>
> — **Sarah Chen, Event Organizer**

1.2 System Requirements

The system requirements for ConveneHub are divided into software and hardware requirements so that the project can be deployed, demonstrated, and extended in a structured manner.

1.2.1 Software Requirements

- Operating System: Windows, Linux, or macOS.
- Frontend: React with Vite, JavaScript, React Router, Tailwind CSS.
- Backend: Node.js, Express.js, npm.
- Database: MongoDB with Mongoose.
- Additional Services: Razorpay, Nodemailer, Google OAuth, Docker.
- Development Tools: VS Code, Git, Postman, browser developer tools.

1.2.2 Hardware Requirements

- Processor: Dual-core or higher.
- RAM: Minimum 8 GB recommended for development.
- Storage: At least 10 GB free space for source code, packages, and assets.
- Internet Connectivity: Required for npm packages, MongoDB Atlas, payment gateway integration, and deployment.

2 Project Architecture

2.1 Technical Architecture

ConveneHub follows a modern client–server architecture built as a monorepo. The frontend is implemented with React and Vite, the backend is implemented using Express.js, and MongoDB provides persistent storage. Authentication, authorization, booking management, payment verification, promoter attribution, and check-in workflows are handled through dedicated API routes and database models.

The technical architecture emphasizes separation of concerns. The client handles presentation, navigation, and user interaction. The server handles business logic, validation, secure workflows, and integration with third-party services. MongoDB stores users, events, bookings, transactions, referrals, and audit-supporting records.

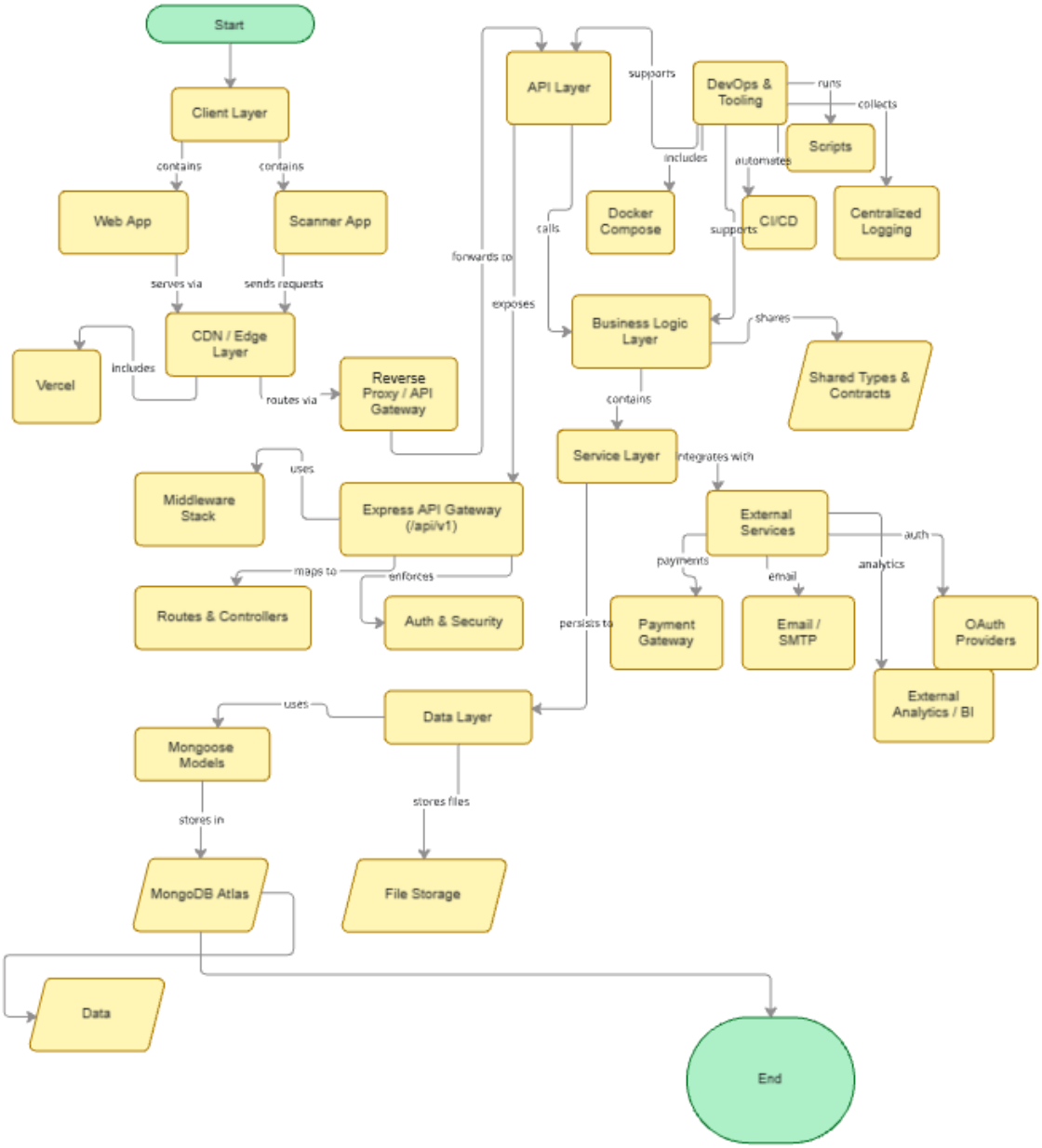


Figure 1: Technical Architecture Diagram

2.2 ER Diagram

The core entities in the system include **User**, **Event**, **Booking**, **PaymentAttempt**, **PromoterProfile**, and related settlement or commission records. Users can create or attend events depending on role, events contain multiple ticket tiers, bookings belong to users and events, and payment records preserve transactional traceability.

Figure 2: Entity Relationship Diagram of ConveneHub

2.3 Key Features

- Role-based authentication and authorization.
- Event creation, publishing, and lifecycle management.
- Free and paid booking workflows.
- Razorpay payment verification.
- QR-based ticket generation and check-in.
- Referral and promoter commission tracking.
- Admin and organizer dashboards with operational visibility.

2.4 Roles and Responsibilities

The platform defines clear operational responsibilities for each stakeholder so that event workflows remain organized, secure, and accountable.

2.4.1 User (Attendee)

- **Registration & Account Management:** Create and maintain a personal account, keep profile and contact information up to date, and secure account credentials.
- **Search & Booking:** Search events, select ticket tiers, and complete bookings while ensuring the accuracy of submitted booking details.
- **Payment & Receipts:** Complete payments using available payment methods, retain receipts for record-keeping, and report payment issues promptly.
- **Attendance & Check-in:** Present the digital ticket or QR code at check-in and follow event entry instructions and terms.
- **Feedback & Communication:** Provide constructive feedback, respond to follow-ups, and use support channels for questions or disputes.
- **Data Privacy:** Consent to applicable data use policies and report suspected account compromise.

2.4.2 Organizer

- **Event Creation & Configuration:** Create and configure events, ticket tiers, pricing, capacity limits, and event metadata such as venue, schedule, and descriptions.
- **Content & Branding:** Provide accurate event descriptions, images, and promotional material while complying with platform content policies.
- **Ticketing & Capacity Management:** Monitor ticket inventory, manage tier availability, and handle waitlists or capacity changes.
- **Attendee Communications:** Send confirmations, reminders, and event-specific communications to attendees in a timely manner.
- **On-site Operations:** Coordinate check-in staff and materials and ensure that access control and QR scanning workflows are followed.
- **Reporting & Reconciliation:** Review event analytics, sales reports, and payment reconciliations and resolve attendee or payment disputes.
- **Compliance & Safety:** Ensure that the event complies with legal, venue, and safety requirements such as permits, age restrictions, and capacity rules.

2.4.3 Promoter

- **Referral Setup:** Register as a promoter and obtain unique referral links or codes for event promotion.
- **Promotion & Ethics:** Market events transparently and ethically while following platform and legal guidelines.
- **Tracking & Reporting:** Monitor referral performance, conversions, and earnings through the promoter dashboard.
- **Payout Coordination:** Provide required payout details and respond to verification requests for commission settlement.
- **Dispute Assistance:** Cooperate with organizers and platform administrators to resolve disputes involving referred sales.

2.4.4 Admin (Platform Administrator)

- **Platform Governance:** Manage tenant onboarding, role provisioning, and high-level platform settings.
- **User & Tenant Management:** Create, suspend, or remove user and tenant accounts and resolve escalated account issues.
- **Compliance & Security Oversight:** Enforce security policies, data protection procedures, and regulatory compliance across the platform.
- **Financial Controls:** Oversee payment flows, commission settlement processes, and financial reporting and audit transactions when needed.
- **System Monitoring & Maintenance:** Monitor system health, logs, and uptime and coordinate maintenance and incident response.
- **Policy & Moderation:** Approve or remove content, handle policy violations, and manage abuse reports.

3 User Flow

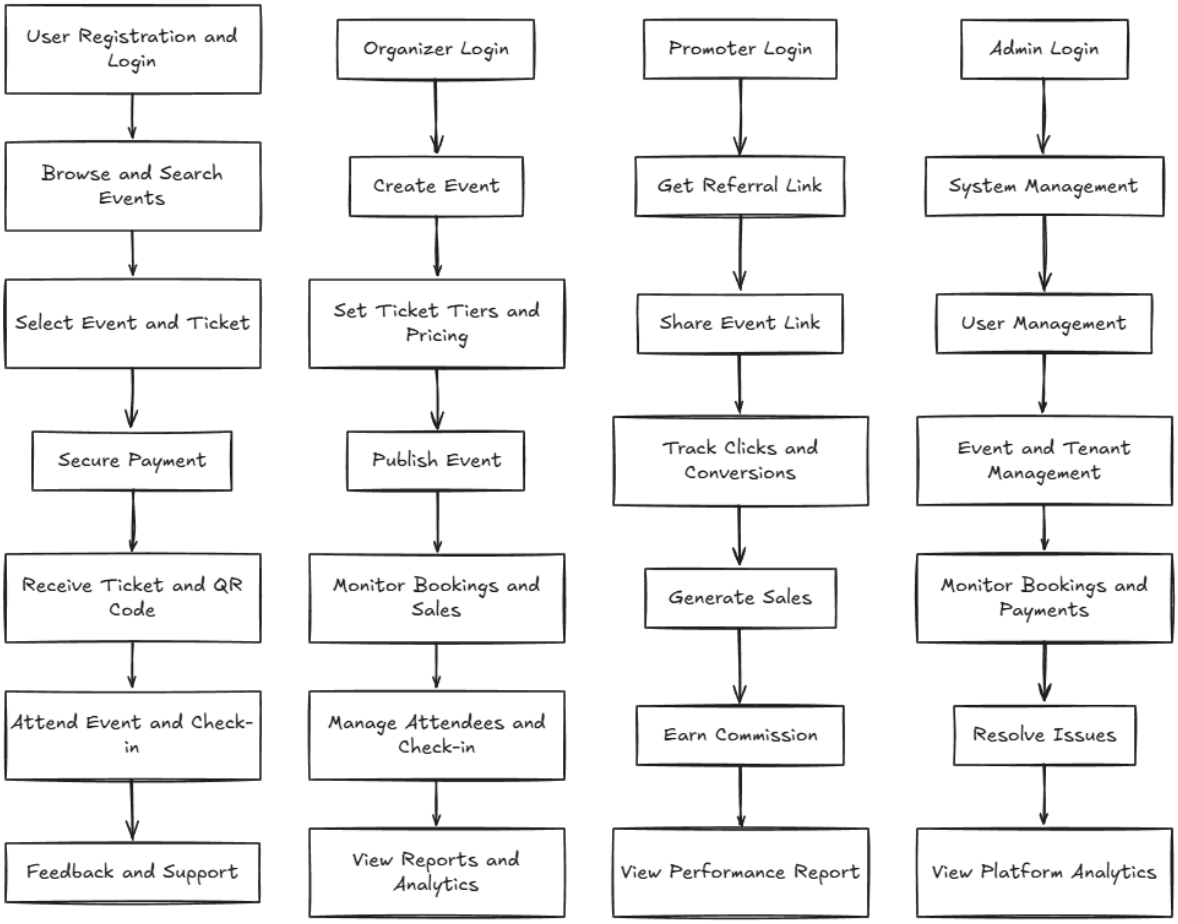


Figure 3: User Flow Overview of ConveneHub

User Flow: Users register, search events, book tickets, make payments, attend events, and provide feedback.

Organizer Flow: Organizers create events, set ticket prices, publish events, track sales, manage check-ins, and view analytics.

Promoter Flow: Promoters share referral links, track clicks and sales, earn commissions, and review performance reports.

Admin Flow: Admins manage users, tenants, events, payments, resolve issues, and monitor platform performance.

4 MVC Pattern

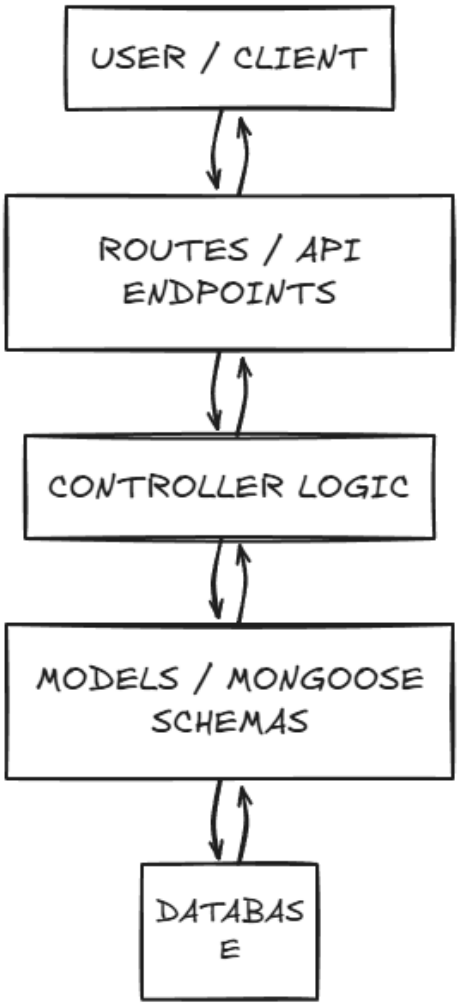


Figure 4: MVC Architecture for ConveneHub

The ConveneHub application follows an MVC-inspired architectural pattern. This approach separates the backend into three main layers: Model, View, and Controller. In ConveneHub, the backend is built using Express, TypeScript, and MongoDB, so the routing layer acts as the View, the route and service logic acts as the Controller, and the Mongoose schemas act as the Model. This separation improves maintainability, scalability, and code organization.

4.1 Model Layer

The Model layer handles all data-related logic in ConveneHub. It defines the structure of the application data using Mongoose schemas and interacts directly with MongoDB. The models store important entities such as users, events, bookings, tickets, attendees, referral links, commissions, and check-ins. These models are found in `backend/src/models` and include files such as `User.ts`, `Event.ts`, `Booking.ts`, `Ticket.ts`, `Attendee.ts`, `ReferralLink.ts`, `Commission.ts`, and `CheckIn.ts`.

4.2 Controller Layer

The Controller layer acts as the middle layer between the routes and the models. It receives requests from the routing layer, processes the input, applies validation or business rules, calls the required model or service methods, and sends a response back to the client. In ConveneHub, much of this logic is handled through the route handlers and service layer. For example, business logic such as referral commission calculation is placed in `backend/src/services/referrals.service.ts`, while the route files in `backend/src/routes` connect requests to the appropriate logic.

4.3 View Layer

In the context of ConveneHub's backend API, the View layer is represented by the routing layer. The route files define the available endpoints and determine how the application responds to HTTP requests such as `GET`, `POST`, `PATCH`, and `DELETE`. These routes are located in `backend/src/routes` and include `auth.routes.ts`, `events.routes.ts`, `bookings.routes.ts`, `checkins.routes.ts`, `admin.routes.ts`, `analytics.routes.ts`, `payments.routes.ts`, `promoters.rout` `tickets.routes.ts`, and `uploads.routes.ts`.

4.4 Advantages of Using MVC in ConveneHub

- **Separation of Concerns:** Each layer has a clear responsibility, which improves readability and maintenance.
- **Scalability:** New features can be added easily by creating new routes, services, and models.
- **Reusability:** Common business logic can be reused across multiple routes and features.
- **Testing:** Models, services, and route handlers can be tested independently.
- **Better Organization:** The codebase stays structured as the project grows.

4.5 Project Structure in ConveneHub

- **Model:** `backend/src/models/*.ts`
- **View:** `backend/src/routes/*.ts`
- **Controller/Business Logic:** `backend/src/services/*.ts` and route handlers
- **Database:** MongoDB through Mongoose

5 Project Setup and Configuration

5.1 Client Setup (Installing React App)

The client side of ConveneHub is built using a modern React-based frontend with Vite and TypeScript. It provides the user interface for event browsing, registration, booking, dashboards, and role-based pages for attendees, organizers, promoters, and admins. The frontend communicates with the Express backend through API calls and handles dynamic rendering, authentication flows, and user interactions.

Setup Steps

1. Create the frontend folder inside the project workspace.
2. Open the frontend folder in the terminal.
3. Initialize the React application using Vite.
4. Install all required dependencies.
5. Start the development server.

```
mkdir frontend
cd frontend
npm create vite@latest . -- --template react-ts
npm install
npm run dev
```

5.2 Server Setup (npm init)

The server side of ConveneHub is implemented as a Node.js and Express application. It handles authentication, event management, bookings, ticket generation, check-ins, analytics, promoter commissions, and admin operations. The server uses MongoDB with Mongoose for database interaction and includes middleware for authentication, error handling, and request processing.

Setup Steps

1. Create the backend folder inside the project workspace.
2. Open the backend folder in the terminal.
3. Initialize the Node.js project using npm.
4. Install the required backend packages such as Express, Mongoose, CORS, JWT, and bcrypt.
5. Start the server in development mode.

```
mkdir backend
cd backend
npm init -y
npm install express mongoose cors dotenv jsonwebtoken bcryptjs
npm install -D typescript ts-node nodemon @types/express @types/node @types/cors @types/jsonwebtoken
    @types/bcryptjs
```

5.3 Summary

The client and server setup together provide a complete full-stack structure for ConveneHub. The frontend handles the user interface and experience, while the backend manages business logic, data storage, and API communication. This separation improves maintainability, scalability, and development efficiency.

6 Backend Development

6.1 Backend Reference Video

A backend walkthrough or demonstration video can be referenced in this section to explain the project structure, routing, middleware chain, and implementation workflow.

https://drive.google.com/drive/folders/108h-JHAW1qIROXJHJONoUy3r8Izv6M7H?usp=drive_link

6.2 Backend Structure

The backend is organized into configuration, middleware, models, routes, services, utilities, scripts, and deployment files. This structure keeps the API modular, improves maintainability, and supports secure multi-role workflows across authentication, bookings, payments, analytics, and event operations.

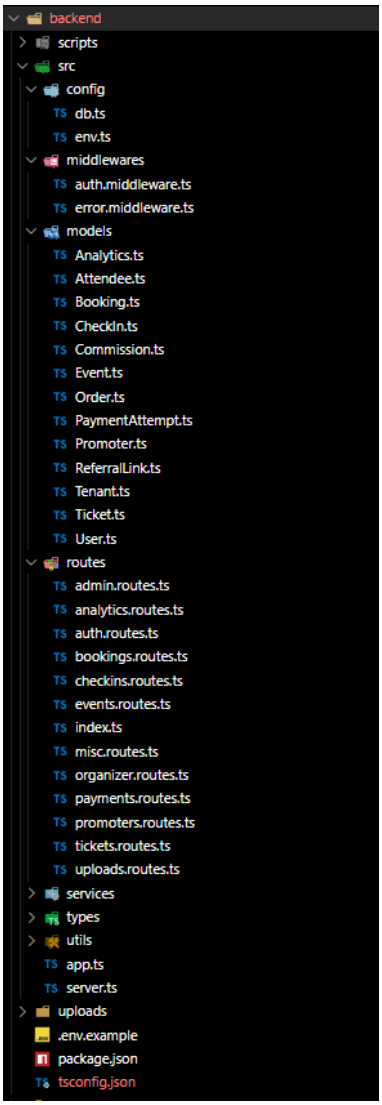


Figure 5: Backend Folder Structure

- **Root**
  - `package.json` — backend dependencies and scripts.
  - `tsconfig.json` — TypeScript configuration for the backend.
  - `server.ts` — server bootstrap that starts the HTTP server.
  - `app.ts` — Express app composition, including middleware and routes.

  - `uploads/` — static uploads root for event images, avatars, and related assets.
  - `scripts/` — utility scripts such as `backfill-role-flow-data.ts`, `migrate-schema-db.ts`, and `smoke-test-api.ts`.
- **src/**
  - **Config**
    - `src/config/env.ts` — environment variable validation and configuration.
    - `src/config/db.ts` — MongoDB and Mongoose connection setup.
  - **Middlewares**
    - `src/middlewares/auth.middleware.ts` — JWT authentication and role checks.
    - `src/middlewares/error.middleware.ts` — centralized error handling.
  - **Models (Mongoose)**
    - `src/models/User.ts` — user schema supporting admin, organizer, promoter, and attendee roles.
    - `src/models/Tenant.ts` — tenant or organization schema.
    - `src/models/Event.ts` — event schema with embedded ticket tiers.
    - `src/models/Booking.ts` — booking and purchase records.
    - `src/models/Ticket.ts` — ticket schema with QR payload and check-in status.
    - `src/models/Attendee.ts` — per-event attendee records.
    - `src/models/ReferralLink.ts` — promoter referral codes and tracking metrics.
    - `src/models/Commission.ts` — promoter commission records.
    - `src/models/CheckIn.ts` — check-in audit trail.
    - `src/models/Analytics.ts` — analytics and metrics model.
    - `src/models/Order.ts` — order records, where applicable.
    - `src/models/PaymentAttempt.ts` — payment transaction history.
    - `src/models/Promoter.ts` — promoter-specific profile model, if present.
  - **Routes**
    - `src/routes/index.ts` — API router aggregator for `/api/v1`.
    - `src/routes/auth.routes.ts` — authentication endpoints such as register, login, and refresh.
    - `src/routes/admin.routes.ts` — admin-only endpoints for tenant and platform management.
    - `src/routes/organizer.routes.ts` — organizer-specific endpoints.
    - `src/routes/events.routes.ts` — event CRUD endpoints.
    - `src/routes/bookings.routes.ts` — booking creation and management.
    - `src/routes/tickets.routes.ts` — ticket retrieval and QR generation.
    - `src/routes/checkins.routes.ts` — QR scanning and check-in endpoints.
    - `src/routes/promoters.routes.ts` — referral link management.
    - `src/routes/payments.routes.ts` — payment gateway actions and webhooks.
    - `src/routes/analytics.routes.ts` — reporting endpoints.
    - `src/routes/uploads.routes.ts` — file upload handling.
    - `src/routes/misc.routes.ts` — health checks and utility endpoints.
  - **Services**
    - `src/services/referrals.service.ts` — referral and commission business logic.
    - Other service modules may include files such as `payments.service.ts` and `events.service.ts`.
  - **Utils / Helpers**
    - `src/utils/token.ts` — JWT creation and verification helpers.

    * `src/utils/tenant.ts` — tenant helper functions.
    * `src/utils/qr.ts` — QR generation utilities.
    * `src/utils/mail.ts` — email sending with Nodemailer.
    * `src/utils/error.ts` — custom error classes.
    * `src/utils/validation.ts` — shared validation helpers.
  - **Types**
    * `src/types/` — TypeScript interfaces such as common types, API types, and database types.

- **uploads/**
  - `uploads/events/` — event images.
  - `uploads/...` — other uploaded assets.
- **scripts/**
  - `scripts/backfill-role-flow-data.ts` — database backfill utilities.
  - `scripts/migrate-schema-db.ts` — migration helpers.
  - `scripts/smoke-test-api.ts` — quick API smoke tests.
- **Deployment & Config**
  - `.env.example` — backend environment variable template.
  - `docker-compose.yml` — local development stack for MongoDB and backend services.
  - `README.md` or docs — running instructions and environment setup.

7 Database Development

7.1 Configure MongoDB

ConveneHub uses MongoDB as its database and Mongoose as the object data modeling library. This combination is a strong fit for the project because ConveneHub stores structured but flexible data such as users, events, bookings, tickets, attendees, referral links, commissions, and check-in records. The database layer supports the full event workflow, from registration to booking, ticket generation, promoter tracking, and attendee check-in.

The first step in database development is to configure MongoDB and connect it to the backend application. In ConveneHub, the MongoDB connection is handled in the backend configuration layer so that all API routes can access the database through a single connection point.

7.2 Create Database Connection

The database connection file is responsible for connecting the Express server to MongoDB using Mongoose. This ensures the application can perform database operations such as creating users, storing events, recording bookings, and updating check-in details.

Database connection code (`backend/src/config/db.ts`):

```
import mongoose from 'mongoose';

export async function connectDatabase(uri: string): Promise<void> {
  mongoose.set('bufferCommands', false);
  await mongoose.connect(uri, {
    serverSelectionTimeoutMS: 4000,
```

```
  });
}
```

7.3 Create Schemas and Models

After setting up the database connection, the next step is to define the schemas and models. In ConveneHub, each major entity is stored in its own Mongoose model. These models define the structure of the data and keep the database organized.

User model:

```
const userSchema = new Schema<UserDocument>(
  {
    fullName: { type: String, required: true, trim: true, alias: 'name' },
    email: { type: String, required: true, unique: true, lowercase: true, trim: true },
    passwordHash: { type: String, required: true, alias: 'password' },
    role: { type: String, enum: ['admin', 'organizer', 'promoter', 'attendee'], default: 'attendee' },
    tenantId: { type: String },
    phone: { type: String },
    city: { type: String },
    emailVerified: { type: Boolean, default: false },
  },
  { timestamps: true }
);

export const UserModel = model<UserDocument>('User', userSchema);
```

This model is used for user registration, login, and role-based access.

Event model:

```
const ticketTierSchema = new Schema<TicketTier>(
  {
    name: { type: String, required: true, trim: true },
    price: { type: Number, required: true, min: 0 },
    quantity: { type: Number, required: true, min: 1 },
    soldCount: { type: Number, default: 0, min: 0 },
  },
  { _id: false }
);
```

The Event model stores details such as title, venue, date, capacity, status, and ticket tiers. It is used by organizers to create and manage events.

Booking model:

```
const bookingSchema = new Schema<BookingDocument>(
  {
    eventId: { type: String, required: true },
    attendeeId: { type: String, required: true, alias: 'userId' },
    tierName: { type: String, required: true },
```

```
    ticketPrice: { type: Number, required: true, min: 0 },
    ticketsCount: { type: Number, required: true, min: 1 },
    amount: { type: Number, required: true, min: 0 },
    bookingCode: { type: String, required: true, unique: true },
    bookingStatus: { type: String, enum: ['confirmed', 'cancelled'], default: 'confirmed' },
    paymentStatus: { type: String, enum: ['pending', 'paid', 'failed', 'refunded'], default: 'paid' },
  },
  { timestamps: true }
);

export const BookingModel = model<BookingDocument>('Booking', bookingSchema);
```

This model stores ticket purchase details and payment status.

7.4 Database Structure in ConveneHub

The database supports the following main collections:

- `users`
- `tenants`
- `events`
- `bookings`
- `tickets`
- `attendees`
- `referrallinks`
- `commissions`
- `checkins`

7.5 Advantages of This Database Design

- **Scalability:** MongoDB handles growing event and booking data efficiently.
- **Flexibility:** Mongoose schemas make it easy to manage changing data structures.
- **Maintainability:** Each entity has a separate model file.
- **Role Support:** The database supports admins, organizers, promoters, and attendees.
- **Performance:** The schema design supports fast bookings, ticket generation, and reporting.

8 Front-End Development

Frontend Reference Video Link:

https://drive.google.com/file/d/1PZM2sgM0Hshucu8Dj-eENS5sHaxrrR9e/view?usp=drive_link

8.1 Frontend Structure

The frontend is organized to separate routing, feature pages, reusable UI components, shared utilities, assets, and deployment configuration. This structure improves maintainability, supports role-based interfaces, and keeps the

application scalable as new modules are introduced.

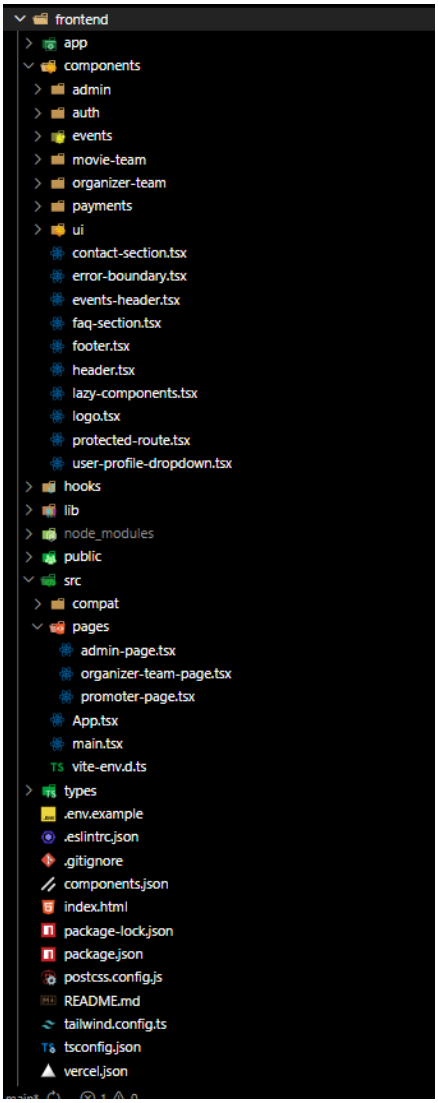


Figure 6: Frontend Folder Structure

- **Root**
  - `index.html` — HTML entry point and React mount target.
  - `package.json` — frontend dependencies and development scripts.
  - `tsconfig.json` — TypeScript compiler configuration.
  - `vite.config.ts` — Vite bundler and development server configuration.
  - `tailwind.config.ts` — Tailwind CSS theme and utility configuration.
  - `postcss.config.js` — PostCSS and Tailwind processing pipeline.
- **Pages / App**
  - `app/layout.tsx` — root layout and global providers.
  - `app/client-layout.tsx` — client-side layout and hydration helpers.
  - `app/page.tsx` — home or landing page.
  - `app/not-found.tsx` — 404 error page.
  - `app/globals.css` — global styles and shared visual overrides.

- **Auth Pages**
  - `app/auth/login/page.tsx` — login form and authentication flow.
  - `app/auth/callback/*` — OAuth callback handlers.
  - `app/auth/error/*` — authentication error handling pages.
  - `app/reset-password/page.tsx` — password reset workflow.
- **Events Pages**
  - `app/events/page.tsx` — event listing and search page.
  - `app/events/[id]/page.tsx` — event details and booking page.
  - `app/events/layout.tsx` — layout for the events section.
  - `app/events/events.css` — event-specific styling.
- **Organizer & Admin Pages**
  - `app/organizer/page.tsx` — organizer dashboard.
  - `app/organizer/*` — event management, ticket tiers, and analytics pages.
  - `app/admin/page.tsx` — administrator dashboard and controls.
  - `app/admin/users/*` — user and tenant management pages.
- **Components**
  - `components/header.tsx` — navigation header.
  - `components/footer.tsx` — site-wide footer.
  - `components/logo.tsx` — logo component.
  - `components/protected-route.tsx` — access control wrapper.
  - `components/error-boundary.tsx` — UI error boundary.
  - `components/user-profile-dropdown.tsx` — user menu and profile actions.
- **Event Components**
  - `components/events/event-card.tsx` — event preview card.
  - `components/events/event-list.tsx` — event grid or list component.
  - `components/events/event-detail.tsx` — event information panel.
  - `components/events/event-booking-page.tsx` — ticket selection and booking flow.
- **UI Primitives (`components/ui`)**
  - `components/ui/button.tsx` — button primitives.
  - `components/ui/input.tsx` — text input with validation states.
  - `components/ui/modal.tsx` — modal and dialog wrapper.
  - `components/ui/card.tsx` — card container.
  - `components/ui/table.tsx` — tabular data display for admin and organizer views.
  - `components/ui/toast.tsx` — notification component.
- **Hooks**
  - `hooks/use-toast.ts` — toast notification hook.
  - Other custom hooks may be colocated inside relevant features as needed.

- **Lib / Utils**
  - `lib/client.ts` — API client with token injection and fetch wrapper logic.
  - `lib/storage.ts` — local and session storage helpers.
  - `lib/utils.ts` — general helper functions such as formatters and date utilities.
  - `lib/hydration-utils.ts` — server-side rendering and hydration helpers.
  - `lib/csv-export.ts` — CSV export utilities.
  - `lib/validation/*` — schema validation helpers.
- **Styles & Assets**
  - `public/logo/*` — logo SVG files and favicon assets.
  - `public/fonts/*` — web font files.
  - `components/*.css` or page-level CSS files — component and page-specific styles.
- **Types**
  - `types/database.types.ts` — shared frontend type definitions such as User, Event, Booking, and Ticket.
  - `shared/types.ts` — contracts shared between frontend and backend, where applicable.
- **Build / Deploy**
  - `vercel.json` — Vercel deployment configuration, if used.
  - `components.json` — project-specific component metadata.

9 Project Resources

Video Demo Link:

https://drive.google.com/drive/folders/1P7KofpS5nRWnlgK51JgYf2iC_t5IJ1Oe

9.1 Deployment and Repository Links

GitHub Repository Link:

https://github.com/lviffy/convenehub

Frontend Deployed Link:

https://convenehub.vercel.app/

Backend Deployed Link:

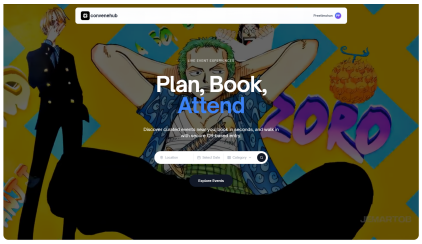
https://convenehub-backend-production.up.railway.app/

9.2 Mock Data Reference

The project includes representative mock data for testing organizer, attendee, promoter, booking, and event-management workflows. A sample mock dataset is shown below for demonstration purposes.


```
{"events":[{"name":"Tech Expo 2026","type":"paid","tiers":[{"name":"General","price":299,"quantity":200},{"name":"VIP","price":799,"
    quantity":50}]}]}
```

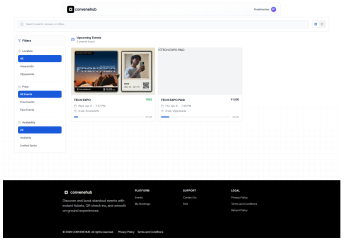
10 Visual References



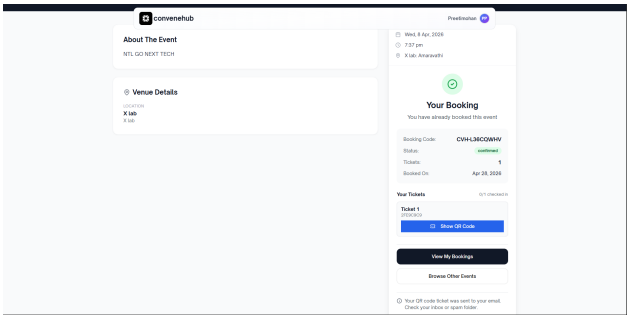
Landing Page

Login Page

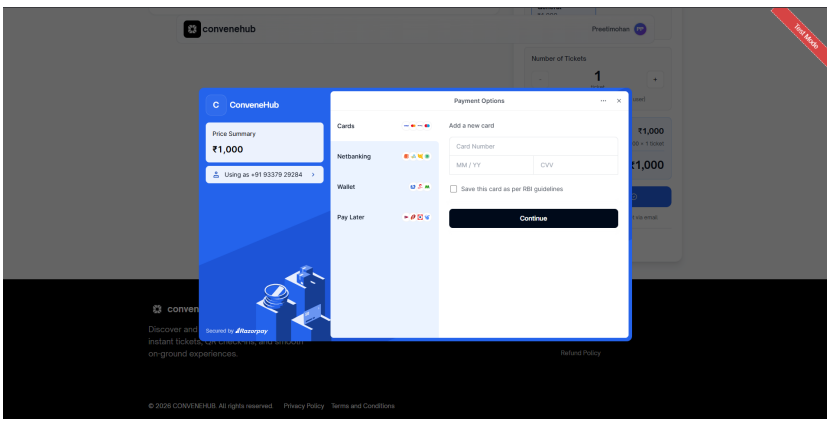
Create Account



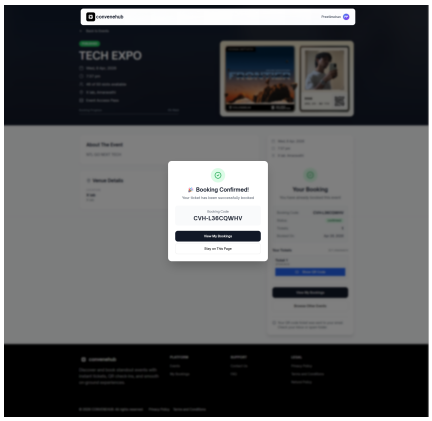
Event Platform



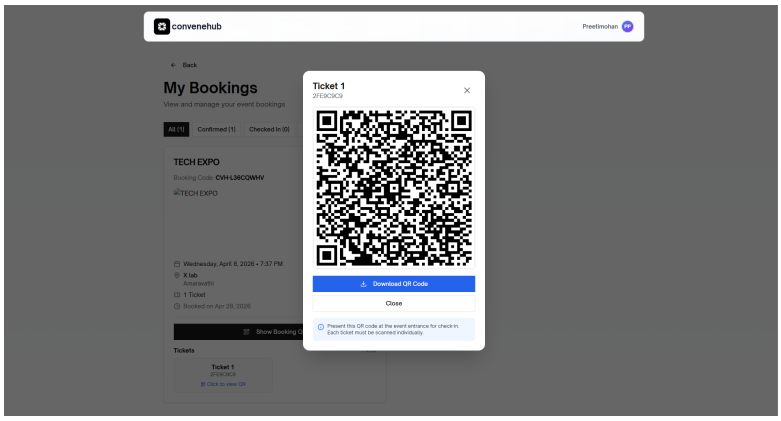
Tech Expo Booking Page



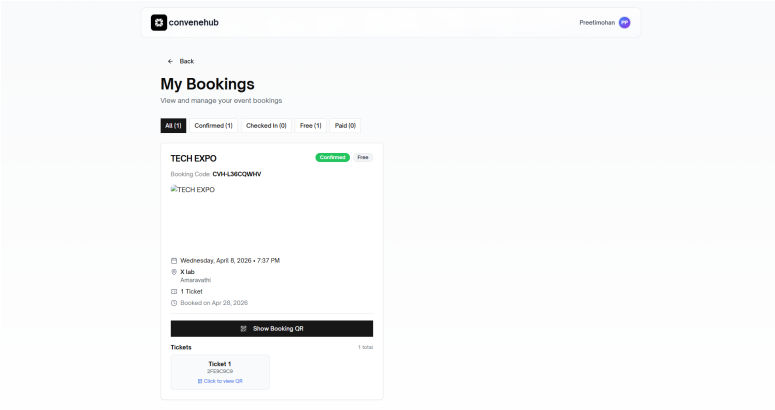
Razor Pay Payment Gateway



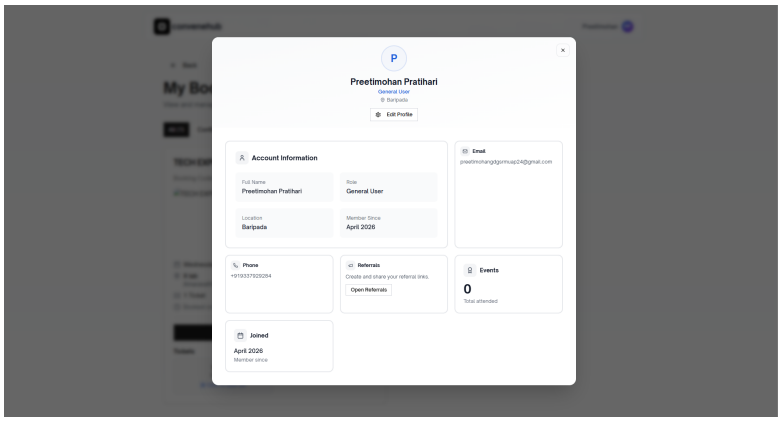
Unique Code for each booking



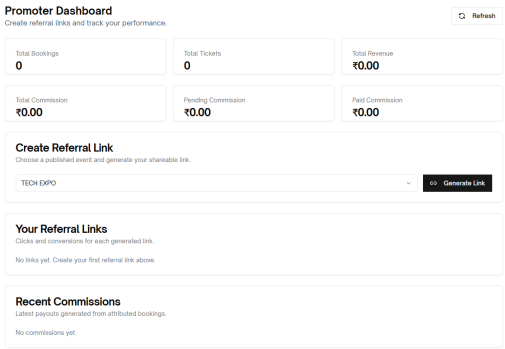
Unique QR for each booking



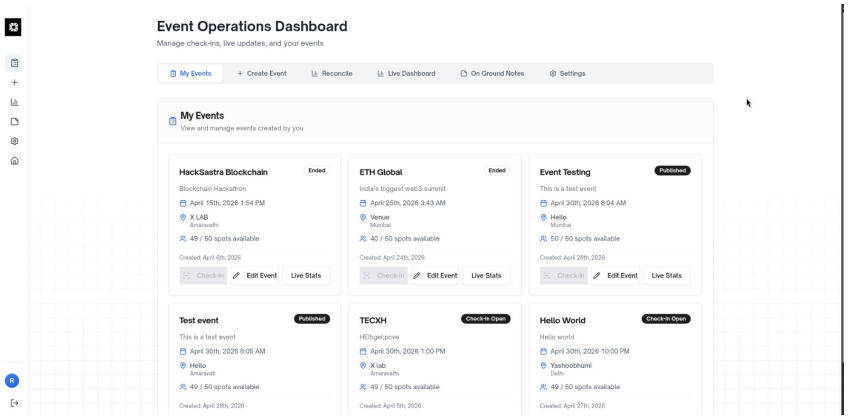
My booking section



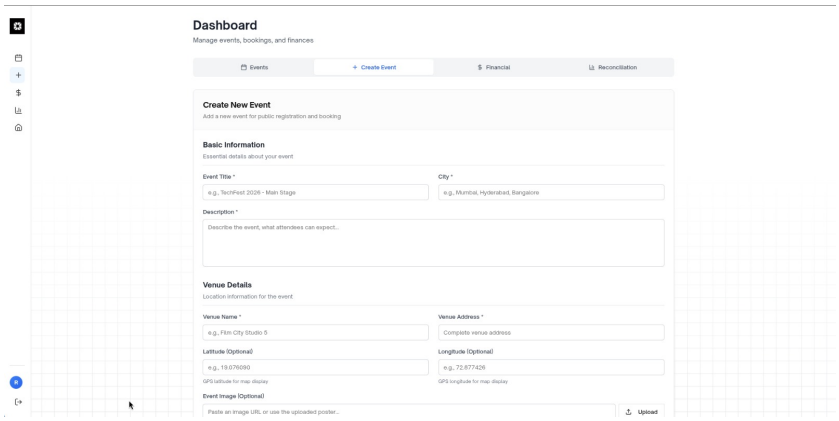
Profile



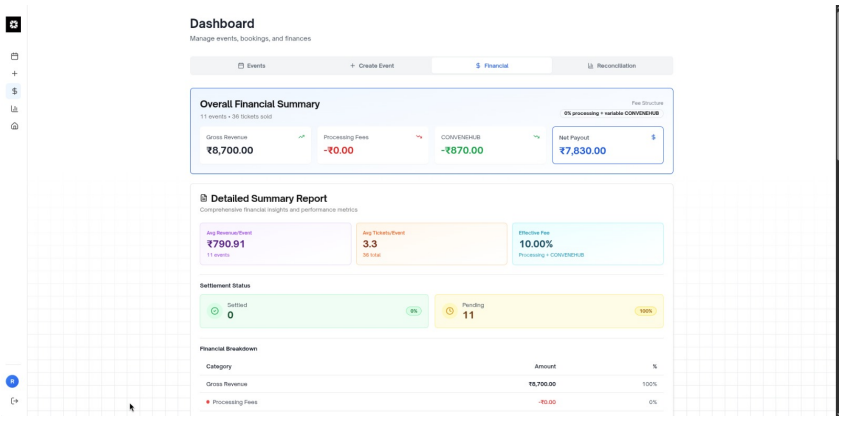
Referal section



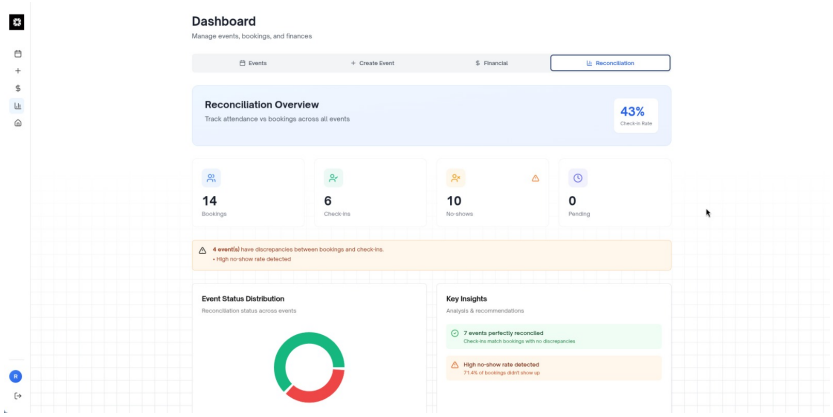
My_Events.jpeg



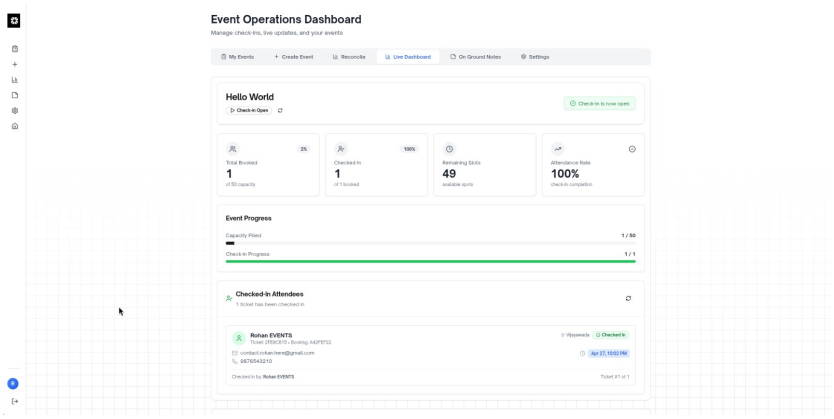
Create Event Admin Portal



FInancial Dashboard Admin Portal



Reconcillation Admin Portal



Live Dashboard Organiser Portal

Bibliography

References

[1] Sandhu, R. S., Coynek, E. J., Feinstein, H. L., & Youman, C. E. (2001). "Role-Based Access Control Models." *IEEE Computer*, 29(2), 38–47.

[2] Brewer, E. A. (2000). "Towards Robust Distributed Systems." In *Proceedings of the Nineteenth Annual ACM Symposium on Principles of Distributed Computing*.

[3] Gray, J., & Lamport, L. (2006). "Consensus on Transaction Commit." *ACM Transactions on Database Systems*, 31(1), 133–160.

[4] Express.js Official Documentation. https://expressjs.com

[5] Mongoose Official Documentation. https://mongoosejs.com

[6] React Documentation. https://react.dev

[7] Razorpay API Documentation. https://razorpay.com/docs

[8] OWASP Top 10 2021. https://owasp.org/Top10/

[9] JSON Web Token (JWT) RFC 7519. https://tools.ietf.org/html/rfc7519

[10] MongoDB Security Best Practices. https://docs.mongodb.com/manual/security/

[11] Statista. (2024). "Event Management Software Market Size." https://www.statista.com

[12] Gartner. (2024). "Event Management Technology Landscape." Gartner Research.

A Appendix

A.1 Database Schema Reference

A.1.1 User Collection

```
{
  _id: ObjectId,
```

```
  email: String,
  passwordHash: String,
  fullName: String,
  role: String,
  tenantId: ObjectId,
  googleId: String,
  otpCode: String,
  otpExpiresAt: Date,
  isVerified: Boolean,
  createdAt: Date,
  updatedAt: Date
}
```

A.1.2 Event Collection

```
{
  _id: ObjectId,
  name: String,
  description: String,
  organizerId: ObjectId,
  tenantId: ObjectId,
  date: Date,
  location: String,
  capacity: Number,
  status: String,
  tiers: [{ name: String, price: Number, quantity: Number, soldCount: Number }],
  imageUrl: String,
  createdAt: Date,
  updatedAt: Date
}
```

A.1.3 Booking Collection

```
{
  _id: ObjectId,
  attendeeId: ObjectId,
  eventId: ObjectId,
  tier: String,
  amount: Number,
  ticketsCount: Number,
  bookingCode: String,
  bookingStatus: String,
  paymentStatus: String,
  paymentMethod: String,
  razorpayPaymentId: String,
  referralCode: String,
  promoterId: ObjectId,
  createdAt: Date,
  updatedAt: Date
}
```

A.2 Environment Configuration Example

```
NODE_ENV=production
PORT=8080
MONGODB_URI=mongodb+srv://user:password@cluster.mongodb.net/convenehub
JWT_ACCESS_SECRET=your-secure-access-secret-key-min-32-chars
JWT_REFRESH_SECRET=your-secure-refresh-secret-key-min-32-chars
FRONTEND_ORIGIN=https://convenehub.vercel.app,https://www.convenehub.in
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_SECURE=false
SMTP_USER=noreply@convenehub.in
SMTP_PASSWORD=your-app-password
GOOGLE_CLIENT_ID=your-client-id.apps.googleusercontent.com
GOOGLE_CLIENT_SECRET=your-client-secret
RAZORPAY_KEY_SECRET=your-razorpay-secret-key
UPLOAD_ROOT=data/uploads
```

A.3 Postman Collection Snippet

```
{
  "info": {
    "name": "ConveneHub API",
    "version": "1.0.0"
  },
  "item": [
    {
      "name": "Authentication",
      "item": [
        {
          "name": "Register",
          "request": {
            "method": "POST",
            "url": "{{base_url}}/api/v1/auth/register"
          }
        }
      ]
    }
  ]
}
```